

Trường Đại học Công nghệ

-Đại học Quốc Gia Hà Nội

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO BÀI TẬP

Môn học: Nhập môn công nghệ số và ứng dụng của trí tuệ nhân tạo

Đề tài: Tác hại của việc lạm dụng Ghi đè phương thức (Method Overriding): Phân tích bài toán Lớp cơ sở mỏng manh (Fragile Base Class)

Sinh viên thực hiện: Nguyễn Quốc Thái

MSV: 25020389

Lớp: IT1

I. MỞ ĐẦU

1. Vấn đề nghiên cứu

Ghi đè phương thức (Method Overriding) là cơ chế cốt lõi để tạo nên tính đa hình (Polymorphism) đồng thời phá vỡ tính đóng gói (encapsulation) nếu không được thiết kế chặt chẽ. Hậu quả nghiêm trọng nhất của việc này được giới học thuật định nghĩa là Bài toán Lớp cơ sở mỏng manh (Fragile Base Class - FBC).

2. Tiêu chí đánh giá độ tin cậy của các nguồn thông tin

Dựa trên 5 tiêu chí gồm: tác giả, cơ quan xuất bản(CQXB), phương pháp nghiên cứu(PP nghiên cứu), trích dẫn, tính cập nhật.

II. BẢNG TỔNG HỢP VÀ ĐÁNH GIÁ ĐỘ TIN CẬY

STT	Tài liệu tham khảo	Phân tích 5 tiêu chí đánh giá độ tin cậy	Phân tích Ưu điểm / Nhược điểm	Xếp hạng tin cậy
1	Mikhajlov & Sekerinski (1998), A study of the fragile base class problem	- Tác giả: Các nhà khoa học máy tính tại ĐH McMaster. - CQXB: Springer (Kỷ yếu hội nghị hàng đầu ECOOP). - PP nghiên cứu: Lập luận toán học, mô hình hóa tĩnh và chứng minh định lý. - Trích dẫn: >250 lượt (chỉ số tốt cho bài báo lý thuyết sâu). - Tính cập nhật: Năm 1998. Dù cũ nhưng đây là tài liệu nền tảng định nghĩa bài toán FBC.	Ưu: Định nghĩa chuẩn xác định mức lý thuyết nền tảng. Nhược: Dùng mã giả (pseudocode), cần tự ánh xạ sang cấu trúc Java hiện đại.	Rất cao
2	Businge et al. (2019), An empirical study of the fragile base class problem in Java libraries	- Tác giả: J. Businge và nhóm nghiên cứu quốc tế. - CQXB: IEEE Transactions on Software Engineering (Tạp chí Q1 danh giá nhất ngành). - PP nghiên cứu: Thực nghiệm định lượng (Empirical) trên hàng nghìn thư viện Java. - Trích dẫn: >50 lượt (đang tăng). - Tính cập nhật: 2019, số liệu phản ánh chính xác hệ sinh thái Java hiện đại.	Ưu: Có số liệu thống kê thực tế về tỷ lệ hỏng hóc hệ thống do ghi đè sai. Nhược: Giới hạn phân tích trong hệ sinh thái Eclipse (OSGi framework).	Tuyệt đối
3	Snyder (1986), Encapsulation and inheritance in object-oriented...	- Tác giả: Alan Snyder (Nhà khoa học tiên phong về OOP). - CQXB: ACM SIGPLAN (Kỷ yếu OOPSLA). - PP nghiên cứu: Phân tích kiến trúc ngôn ngữ đối sánh. - Trích dẫn: >2800 lượt (Cực kỳ cao). - Tính cập nhật: 1986. Là bài báo kinh điển vạch ra lỗ hổng của kế thừa.	Ưu: Chứng minh đanh thép việc ghi đè làm phá vỡ ranh giới đóng gói. Nhược: Không đề cập trực tiếp đến Java (ngôn ngữ chưa ra đời lúc đó).	Tuyệt đối
4	Tempero et al. (2013), What programmers do with	- Tác giả: E. Tempero (ĐH Auckland) và cộng sự. - CQXB: Springer (Kỷ yếu ECOOP). - PP nghiên cứu: Phân tích mã tĩnh (Static analysis) trên bộ Qualitas Corpus (hàng triệu	Ưu: Chỉ ra các anti-pattern phổ biến mà lập trình viên gặp phải khi lạm dụng ghi đè. Nhược: Chỉ nêu hiện trạng thống	Cao

	inheritance in Java	dòng code). - Trích dẫn: >150 lượt. - Tính cập nhật: 2013. Dữ liệu thực tiễn vẫn đúng với Java 8+.	kê, không đề xuất giải pháp thiết kế sâu.	
5	Ruby & Leavens (2000), Safely creating correct subclasses...	- Tác giả: G.T. Leavens (Chuyên gia cha đẻ của Java Modeling Language). - CQXB: ACM (Kỹ yếu OOPSLA). - PP nghiên cứu: Đề xuất mô hình đặc tả hành vi. - Trích dẫn: >150 lượt. - Tính cập nhật: 2000. Kiến trúc hợp đồng (Design by Contract) không đổi.	Ưu: Đưa ra phương pháp viết lớp con an toàn mà không cần đọc mã lớp cha.Nhược: Lý thuyết JML phức tạp để áp dụng vào dự án thực tế thông thường.	Cao
6	Taivalsaari (1996), Classes vs. prototypes...	- Tác giả: Antero Taivalsaari (Kỹ sư lỗi Sun Microsystems). - CQXB: Tạp chí IEEE Xplore. - PP nghiên cứu: Phân loại học và quan sát lịch sử. - Trích dẫn: >900 lượt. - Tính cập nhật: 1996. Mang tính đối chiếu các paradigm.	Ưu: Đặt kế thừa lớp lên bàn cân, làm rõ điểm yếu của ghi đề đa hình.Nhược: Không trực tiếp giải quyết vấn đề bằng code.	Khá
7	Bloch (2018), Effective Java (3rd Edition)	- Tác giả: Joshua Bloch (Kiến trúc sư trưởng thiết kế Java Collections API). - CQXB: Addison Wesley Professional. - PP nghiên cứu: Rút từ kinh nghiệm thực tiễn (Best practices) phát triển JDK lỗi. - Trích dẫn: >6000 lượt. - Tính cập nhật: 2018. Tương thích hoàn toàn Java hiện đại (Java 9+).	Ưu: Có mã nguồn HashSet thực tế minh họa trực quan sự nguy hiểm của FBC.Nhược: Là sách hướng dẫn thực hành, không đi kèm kiểm chứng số liệu diện rộng.	Tuyệt đối
8	Martin (2017), Clean Architecture	- Tác giả: Robert C. Martin (Uncle Bob) - Tác giả nguyên lý SOLID). - CQXB: Prentice Hall. - PP nghiên cứu: Tổng hợp lý thuyết thiết kế kiến trúc. - Trích dẫn: >2500 lượt. - Tính cập nhật: 2017. Sách gối đầu giường về kiến trúc hệ thống hiện nay.	Ưu: Nâng tầm tác hại của ghi đề sai từ mức class lên mức làm sập toàn bộ module kiến trúc.Nhược: Tính trừu tượng cao, ít code Java cấp thấp.	Rất cao
9	Oracle (2024), The Java™ Tutorials - Polymorphism	- Tác giả: Đội ngũ phát triển Java Core. - CQXB: Oracle Corporation. - PP nghiên cứu: Đặc tả kỹ thuật chính thức. - Trích dẫn: Nguồn tham chiếu gốc. - Tính cập nhật: Cập nhật liên tục theo các bản và Java mới nhất.	Ưu: Giải thích chính xác luật của trình biên dịch và giải pháp dùng từ khóa final.Nhược: Chỉ nêu "cách làm", không bàn luận sâu về "triết lý thiết kế".	Đáng tin cậy
10	Fowler (2003), Public versus published interfaces	- Tác giả: Martin Fowler (Chuyên gia hàng đầu về Refactoring). - CQXB: IEEE Software. - PP nghiên cứu: Quan sát công nghiệp (Industrial observation). - Trích dẫn: >300 lượt. - Tính cập nhật: 2003. Khái niệm quản lý	Ưu: Làm rõ lý do kiến trúc tại sao không nên cho phép ghi đề tự do trong các thư viện mở.Nhược: Dạng bài bình luận ngắn (column).	Cao

III. KẾT LUẬN

Quá trình nghiên cứu và phân tích các tài liệu học thuật cho thấy, mặc dù ghi đè phương thức (Method Overriding) là công cụ đắc lực để triển khai tính đa hình trong Java, việc lạm dụng cơ chế này thiếu kiểm soát sẽ mang lại rủi ro lớn cho kiến trúc phần mềm. Tác hại cốt lõi của việc sử dụng không đúng đắn phương thức ghi đè là sự xâm phạm ranh giới đóng gói (encapsulation), dẫn trực tiếp đến bài toán Lớp cơ sở mỏng manh (Fragile Base Class). Cơ chế đa hình lúc chạy khiến các lớp con bị phụ thuộc chặt chẽ vào chi tiết cài đặt ẩn bên trong của lớp cha, làm cho hệ thống trở nên giòn vỡ, dễ phát sinh lỗi đệ quy hoặc sai lệch logic mỗi khi thư viện gốc được cập nhật.

Để đảm bảo tính bền vững và khả năng mở rộng của phần mềm, các nghiên cứu chuyên sâu và tài liệu chuẩn mực đều thống nhất khuyến nghị áp dụng triết lý: "Thiết kế và lập tài liệu rõ ràng cho việc kế thừa, nếu không thì phải cấm nó" (bằng cách sử dụng từ khóa "final" cho lớp hoặc phương thức). Đồng thời, nguyên tắc thiết kế "Ưu tiên Tổng hợp hơn Kế thừa" (Composition over Inheritance) cần được áp dụng như một tiêu chuẩn mặc định để thay thế cho việc ghi đè phương thức trong các trường hợp không thực sự cần thiết phải thiết lập mối quan hệ kiểu phụ (Subtyping).

IV. DANH MỤC TÀI LIỆU THAM KHẢO

1. Cơ sở dữ liệu học thuật (Google Scholar, Microsoft Academic, CSDL của thư viện trường)

- **Mikhajlov, L. và Sekerinski, E.**, 1998. A study of the fragile base class problem. Trong: *ECOOP'98—Object-Oriented Programming*. Springer Berlin Heidelberg, tr. 355-369.
- **Ruby, C. và Leavens, G.T.**, 2000. Safely creating correct subclasses without seeing superclass code. Trong: *Proceedings of the 15th ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications (OOPSLA)*. tr. 208-228.
- **Snyder, A.**, 1986. Encapsulation and inheritance in object-oriented programming languages. Trong: *Proceedings of the 11th ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications (OOPSLA)*. tr. 38-45.
- **Tempero, E., Yang, H. và Noble, J.**, 2013. What programmers do with inheritance in Java. Trong: *ECOOP 2013—Object-Oriented Programming*. Springer Berlin Heidelberg, tr. 577-601.

2. Tạp chí khoa học chuyên ngành

- **Businge, J., Openja, M., Nadi, S., Ribeiro, E. và Baumann, C.**, 2019. An empirical study of the fragile base class problem in Java libraries. *IEEE Transactions on Software Engineering*, 47(11), tr. 2372-2387.
- **Fowler, M.**, 2003. Public versus published interfaces. *IEEE Software*, 20(2), tr. 18-19.
- **Taivalsaari, A.**, 1996. Classes vs. prototypes: Some philosophical and historical observations. *Journal of Object-Oriented Programming*, 10(4), tr. 44-50.

3. Sách chuyên khảo

- **Bloch, J.**, 2018. *Effective Java*. Ấn bản lần 3. Boston: Addison-Wesley Professional.
- **Martin, R.C.**, 2017. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston: Prentice Hall.

4. Các nguồn mở trên internet

- **Baeldung**, 2023. *Composition vs. Inheritance in Java*.
- **Oracle**, 2024. *The Java™ Tutorials - Polymorphism*.